Lambda calculus (red highlight good to practice)

<expression> := <name> | <function> | <application>
<function> := λ <name> . <expression>
<application> := <expression><expression>

In the slides, <function> is <abs>
<name> is <var>
Expression for short: E_1, E_2, etc.

Bound & free variables
Free: meaning derived externally
Bound: meaning derived from binding occurrence

$\lambda x.y$ y is free
$\lambda x.x$ x is bound
↑ binding occurrence
↑ bound variable occurrence.

$(\lambda x.x)\ y$ is this being applied to y.

Can be expressed as

$(\lambda x.x)y = x[y/x] = y$

replace occurrences of x with y

$(\lambda x.x)(\lambda y.y\ x)$
Bound Bound Free

General form of substitution: $M[N/x]$
in expression M, replace any instances of x with N

A variable x is free if:

1.) x is free in x

2.) x is free in $\lambda \langle name \rangle . \langle exp \rangle$ if $\langle name \rangle \neq x$ and x is free in $\langle exp \rangle$.

3.) x is free in $E_1 E_2$ if x is free in E_1 or E_2.

A variable x is bound if:

1.) x is bound in $\lambda \langle name \rangle . \langle exp \rangle$ if $\langle name \rangle = x$ or x is bound in $\langle exp \rangle$.

2.) x is bound in $E_1 E_2$ if x is bound in E_1 or E_2

$(\lambda x . x y)(\lambda y . y)$ note that y is both free & bound.

Applications and substitutions

$(\lambda x . x)(\lambda x . x) \Rightarrow (\lambda x . x)(\lambda z . z)$

we can rename, as second x was unbound to the first one.

Then: $[\lambda z . z / x] x = \lambda z . z$ = identity function.

$(\lambda x . (\lambda y . x y)) y$

to avoid accidentally binding the second y, we can rename the first y.

$(\lambda x . (\lambda t . x t)) y = (\lambda t . x t)[y / x] = \lambda t . y t$

Another example

$(\lambda x.(\lambda y.(x(\lambda x.xy))))y$

Rename this y to avoid binding this y.

$(\lambda x.(\lambda t.(x(\lambda x.xt))))y$

$(\lambda t.(x(\lambda x.xt)))[y/x] = (\lambda t.(y(\lambda x.xt)))$

These processes of renaming and substitution are respectively known as α-equivalence and β-reduction.

When doing these, it is imperative we apply each with care to ensure we don't bind free variables by accident.

$\lambda y.xy \neq_\alpha \lambda y.zy$ this is illegal! we can't rename free variables.

$\lambda y.xy \neq_\alpha \lambda x.xx$ this is illegal! it binds the free occurrence of x.

A general rule for α-equivalence:

$\lambda x.E =_\alpha \lambda y.E[y/x]$ where y is not a free variable in E.

η

Eta reduction helps remove redundant lambdas

$\lambda x.Ex \rightarrow E$ if x has no free occurrences in E.

$(\lambda x.\lambda y.xy)(\lambda x.xy)(\lambda x.\lambda y.xy)$

$(\lambda x.x)(\lambda x.xy)(\lambda x.\lambda y.xy)$ η-reduced term 1.

$(\lambda x.x)(\lambda x.xy)(\lambda x.x)$ η-reduced term 3.

$(\lambda x.x)(xy[(\lambda x.x)/x])$ β-reduced.

$(\lambda x.x)((\lambda x.x)y)$ identity function.

$(\lambda x.x)(y)$ identity function.

y

Normal forms:

A β-normal form is achieved when your expression cannot be further β-reduced.

$(\lambda x.xy)(\lambda z.z)t$

$(\lambda x.xy)(t)$

ty ← β-normal form

Worth noting a β-normal form does not always exist, and when they do, there is one unique one.

There are two different reduction strategies:
1.) Normal order reduction
2.) Applicative order reduction

NOR will choose the leftmost and outermost redex (reducible expression), which is not contained within any other redex.
AOR also chooses the leftmost redex, but chooses the innermost one, ie doesn't contain any redexes itself.

NOR guarantees a reduction to β-nf.

An example of NOR vs AOR, where the highlight shows which expression is evaluated

1.) $(\lambda x.(\lambda y.y)x)((\lambda z.z)a)$
$(\lambda y.y)((\lambda z.z)a)$ $\quad (\lambda y.y)x[(\lambda z.z)a/x]$
$(\lambda z.z)a$ $\quad y[(\lambda z.z)a/y]$
a

2.) $(\lambda x.(\lambda y.y)x)((\lambda z.z)a)$
$(\lambda x.x)((\lambda z.z)a)$ $\quad y[x/y]$
$(\lambda x.x)(a)$ $\quad z[a/z]$
a

Some functions are common.

1.) Identity: $(\lambda x.x)M = M$ for any M.
Example: $(\lambda x.x)a = a$

2.) Selection: $(\lambda x.\lambda y.x)MN$
$= (\lambda y.M)N$
$= M$

x: will give first arg.
y: will give second arg.

3.) Constant function: $\lambda x.\lambda y.x = K$
will always return the same thing regardless of arguments.

$(\lambda x.0)M \rightarrow 0$
$(\lambda x.n)M \rightarrow n$

4.) Composition: $\lambda f.\lambda g.\lambda x.f(gx)MNa$

$(\lambda g.\lambda x M(gx))Na$
$(\lambda x.M(Nx))a$
$M(Na)$

5.) Self-application: $\lambda x.xx$

Church encoding

We can represent Booleans similarly to first & second selection functions:

$\lambda x.\lambda y.x$ = True
$\lambda x.\lambda y.y$ = False

if statements are represented as follows:
a b c [if a then b else c]

An example: if true then b else c.

$(\lambda x.\lambda y.x)\,b\,c$
$= (\lambda y.b)\,c$
$= b$

false

$(\lambda x.\lambda y.y)\,b\,c$
$= (\lambda y.y)\,c$
$= c$

A definition for not x:
if x then false else true
$\lambda x.x$ false true
$\lambda x.x\,(\lambda x.\lambda y.y)(\lambda x.\lambda y.x)$

not true:
$(\lambda x.x$ false true$)$ true
$=$ true false true
$= (\lambda x.\lambda y.x)$ false true
$= (\lambda y.$ false$)$ true
$=$ false

a and b: if a then b else false

$(\lambda a.a)(\lambda b.b)$ false

true and false:

= true false false

$(\lambda x.\lambda y.x)$ (false)(false)
$(\lambda y.$ false$)$(false)
false

true and true:

= true true false

$(\lambda x.\lambda y.x)$ (true)(false)
$(\lambda y.$ true$)$ (false)
true

true and not false:

= (true)(not false)(true)
= (true) ((λx.x false true) false)(true)
= (true) (false false true) (true)
= (true) ((λx.λy.y) false true)(true)
= (true)((λy.y) true) (true)
= (true) (true) (true)
= (λx.λy.x) (true) (true)
= (λy.true) (true)
= true

a or b: if a then true else b

$\lambda x.\lambda y.\ x\ true\ y$

false or true :

$= (\lambda x.\lambda y.\ x\ true\ y)\ false\ true$
$= (\lambda y.\ false\ true\ y)\ true$
$=\ false\ true\ true$
$=\ \lambda x.\lambda y.\ y\ true\ true$
$=\ \lambda y.\ y\ true$
$=\ true$

Natural number encoding:

0 is encoded as $\lambda f.\lambda y.\ y$ [note this is = false]
1 is encoded as $\lambda f.\lambda y.\ fy$
2 is encoded as $\lambda f.\ \lambda y.\ f(fy)$
3 is encoded as $\lambda f.\ \lambda y.\ f(f(fy))$
and so on.

A successor function can be defined as follows:
$\lambda z.\lambda f.\lambda y.\ f\ (z\ f\ y)$

succ (1)
[example continued on next page]

Succ (1):

$(\lambda z.\lambda f.\lambda y.f(zfy))(\lambda f.\lambda y.fy)$

$\lambda f.\lambda y.f((\lambda f.\lambda y.fy)fy)$

$\lambda f.\lambda y.f((\lambda y.fy)y)$

$\lambda f.\lambda y.f(fy)$ ← equal to definition of 2

Addition:

$x + y$ is the application of the successor function x times, to y.

A common expression:

$\lambda M.\lambda N.\lambda x.\lambda y.(Mx)((Nx)y)$

$(+ A B) = \lambda x.\lambda y.(Ax)((Bx)y)$

$(+ 2 2) = \lambda x.\lambda y.(2x)((2x)y)$

$= \lambda x.\lambda y.(2x)((2x)y)$

$= \lambda x.\lambda y.(2x)(((\lambda f.\lambda y.f(fy))x)y)$

$= \lambda x.\lambda y.(2x)((\lambda y.x(xy))y)$

$= \lambda x.\lambda y.(2x)(x(xy))$

$= \lambda f.\lambda y.(2f)(f(fy))$

$= \lambda f.\lambda y.((\lambda f.\lambda y.f(fy))f)(f(fy))$

$= \lambda f.\lambda y.(\lambda y.f(fy))(f(fy))$

$= \lambda f.\lambda y.f(f(f(fy)))$ ← definition of 4.

Multiplication:

$\lambda M.\lambda N.\lambda x.(M(Nx))$

$(*A\ B) = \lambda x.(A\ (B\ x))$

$(*2\ 2) = \lambda x.(2(2x))$